

```
In [7]: import pandas as pd
df_train=pd.read_csv("fraudTrain.csv")
df_test=pd.read_csv("fraudTest.csv")
print(df_train.shape,df_test.shape)
print(df_train.columns.tolist())
print(df_train.head(3))
```

(1296675, 23) (555719, 23)

['Unnamed: 0', 'trans_date_trans_time', 'cc_num', 'merchant', 'category', 'amt', 'first', 'last', 'gender', 'street', 'city', 'state', 'zip', 'lat', 'long', 'city_pop', 'job', 'dob', 'trans_num', 'unix_time', 'merch_lat', 'merch_long', 'is_fraud']

	Unnamed: 0	trans_date_trans_time	cc_num	merchant	category	amt	first	last
0	0	2019-01-01 00:00:18	2703186189652095					
1	1	2019-01-01 00:00:44	630423337322					
2	2	2019-01-01 00:00:51	38859492057661					

	merchant	category	amt	first	last
0	fraud_Rippin, Kub and Mann	misc_net	4.97	Jennifer	Bank
1	fraud_Heller, Gutmann and Zieme	grocery_pos	107.23	Stephanie	Gil
2	fraud_Lind-Buckridge	entertainment	220.11	Edward	Sanche

	gender	street	...	lat	long	city_pop
0	F	561 Perry Cove	...	36.0788	-81.1781	3495
1	F	43039 Riley Greens Suite 393	...	48.8878	-118.2105	149
2	M	594 White Dale Suite 530	...	42.1808	-112.2620	4154

	job	dob
0	Psychologist, counselling	1988-03-09
1	Special educational needs teacher	1978-06-21
2	Nature conservation officer	1962-01-19

	trans_num	unix_time	merch_lat	merch_long
0	0b242abb623afc578575680df30655b9	1325376018	36.011293	-82.048315
1	1f76529f8574734946361c461b024d99	1325376044	49.159047	-118.186462
2	ala22d70485983eac12b5b88dad1cf95	1325376051	43.150704	-112.154481

	is_fraud
0	0
1	0
2	0

[3 rows x 23 columns]

```
In [3]: !pip install pandas
```

Requirement already satisfied: pandas in c:\users\vandana prajapati\appdata\local\programs\python\python313\lib\site-packages (3.0.6)
 Requirement already satisfied: numpy>=1.26.0 in c:\users\vandana prajapati\appdata\local\programs\python\python313\lib\site-packages (from pandas) (2.5.3)
 Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\vandana prajapati\appdata\local\programs\python\python313\lib\site-packages (from pandas) (2.9.0.post0)
 Requirement already satisfied: tzdata in c:\users\vandana prajapati\appdata\local\programs\python\python313\lib\site-packages (from pandas) (2026.4)
 Requirement already satisfied: six>=1.5 in c:\users\vandana prajapati\appdata\local\programs\python\python313\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)

[notice] A new release of pip is available: 25.1.1 -> 26.2.1

[notice] To update, run: python.exe -m pip install --upgrade pip

Explore & clean the data

```
In [8]: print(df_train['is_fraud'].value_counts())
        print(df_train['is_fraud'].value_counts(normalize=True)*100)
```

```
is_fraud
0      1289169
1         7506
Name: count, dtype: int64
is_fraud
0      99.421135
1       0.578865
Name: proportion, dtype: float64
```

```
In [7]: print(df_train.isnull().sum())
```

```
Unnamed: 0      0
trans_date_trans_time  0
cc_num          0
merchant        0
category        0
amt             0
first           0
last            0
gender          0
street          0
city            0
state           0
zip             0
lat             0
long            0
city_pop        0
job             0
dob             0
trans_num       0
unix_time       0
merch_lat       0
merch_long      0
is_fraud        0
dtype: int64
```

```
In [9]: print(df_train.dtypes)
```

```
Unnamed: 0          int64
trans_date_trans_time  str
cc_num              int64
merchant            str
category            str
amt                 float64
first               str
last                str
gender              str
street              str
city                str
state               str
zip                 int64
lat                 float64
long                float64
city_pop            int64
job                 str
dob                 str
trans_num           str
unix_time           int64
merch_lat            float64
merch_long           float64
is_fraud             int64
dtype: object
```

```
In [11]: print(df_train.duplicated().sum())
print(df_train['trans_num'].duplicated().sum())
```

```
0
0
```

```
In [10]: print(df_train['amt'].describe())
```

```
count    1.296675e+06
mean      7.035104e+01
std       1.603160e+02
min       1.000000e+00
25%       9.650000e+00
50%       4.752000e+01
75%       8.314000e+01
max       2.894890e+04
Name: amt, dtype: float64
```

```
In [11]: #convert timestamp columns to proper datetime
```

```
df_train['trans_date_trans_time'] = pd.to_datetime(df_train['trans_date_trans_time'])
df_test['trans_date_trans_time'] = pd.to_datetime(df_test['trans_date_trans_time'])

df_train['dob'] = pd.to_datetime(df_train['dob'])
df_test['dob'] = pd.to_datetime(df_test['dob'])

# Drop the useless index column
df_train.drop(columns=['Unnamed: 0'], inplace=True)
df_test.drop(columns=['Unnamed: 0'], inplace=True)
```

```

# Create age feature at time of transaction
df_train['age'] = (df_train['trans_date_trans_time'] - df_train['dob']).dt.days
df_test['age'] = (df_test['trans_date_trans_time'] - df_test['dob']).dt.days

#Extract time-based features (useful for network/velocity features later)

df_train['trans_hour'] = df_train['trans_date_trans_time'].dt.hour
df_train['trans_day'] = df_train['trans_date_trans_time'].dt.dayofweek # 0=
df_train['trans_month'] = df_train['trans_date_trans_time'].dt.month

df_test['trans_hour'] = df_test['trans_date_trans_time'].dt.hour
df_test['trans_day'] = df_test['trans_date_trans_time'].dt.dayofweek
df_test['trans_month'] = df_test['trans_date_trans_time'].dt.month

# cc_num and zip are identifiers, not numbers – treat as string/category
df_train['cc_num'] = df_train['cc_num'].astype(str)
df_test['cc_num'] = df_test['cc_num'].astype(str)

df_train['zip'] = df_train['zip'].astype(str)
df_test['zip'] = df_test['zip'].astype(str)

print(df_train.shape, df_test.shape)
print(df_train[['trans_date_trans_time', 'dob', 'age', 'trans_hour', 'trans_

```

```

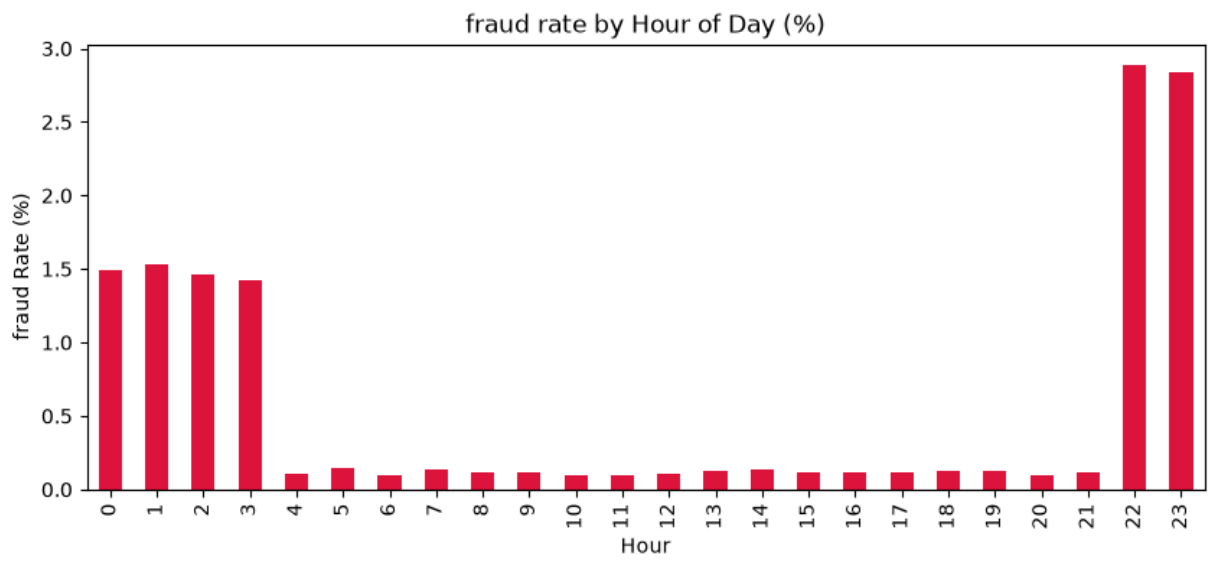
(1296675, 26) (555719, 26)
   trans_date_trans_time  dob  age  trans_hour  trans_day  trans_month
0  2019-01-01 00:00:18 1988-03-09   30           0           1           1
1  2019-01-01 00:00:44 1978-06-21   40           0           1           1
2  2019-01-01 00:00:51 1962-01-19   56           0           1           1
3  2019-01-01 00:01:16 1967-01-12   52           0           1           1
4  2019-01-01 00:03:06 1986-03-28   32           0           1           1

```

```

In [14]: import matplotlib.pyplot as plt
fraud_by_hour=df_train.groupby('trans_hour')['is_fraud'].mean()*100
plt.figure(figsize=(10,4))
fraud_by_hour.plot(kind='bar',color='crimson')
plt.title('fraud rate by Hour of Day (%)')
plt.xlabel('Hour')
plt.ylabel('fraud Rate (%)')
plt.show()

```



In [13]: !pip install matplotlib seaborn networkx scikit-learn

```

Collecting matplotlib
  Downloading matplotlib-3.11.2-cp313-cp313-win_amd64.whl.metadata (80 kB)
Collecting seaborn
  Downloading seaborn-0.13.2-py3-none-any.whl.metadata (5.4 kB)
Collecting networkx
  Downloading networkx-3.7-py3-none-any.whl.metadata (6.7 kB)
Collecting scikit-learn
  Downloading scikit_learn-1.9.1-cp313-cp313-win_amd64.whl.metadata (9.3 kB)
Collecting contourpy>=1.0.1 (from matplotlib)
  Downloading contourpy-1.4.0-cp313-cp313-win_amd64.whl.metadata (3.8 kB)
Collecting cycler>=0.10 (from matplotlib)
  Downloading cycler-0.12.1-py3-none-any.whl.metadata (3.8 kB)
Collecting fonttools>=4.28.2 (from matplotlib)
  Downloading fonttools-4.66.0-cp313-cp313-win_amd64.whl.metadata (131 kB)
Collecting kiwisolver>=1.3.1 (from matplotlib)
  Downloading kiwisolver-1.5.1-cp313-cp313-win_amd64.whl.metadata (5.2 kB)
Requirement already satisfied: numpy>=1.25 in c:\users\vandana prajapati\app
data\local\programs\python\python313\lib\site-packages (from matplotlib) (2.
5.3)
Requirement already satisfied: packaging>=20.0 in c:\users\vandana prajapati
\appdata\local\programs\python\python313\lib\site-packages (from matplotlib)
(25.0)
Collecting pillow>=9 (from matplotlib)
  Downloading pillow-12.3.0-cp313-cp313-win_amd64.whl.metadata (9.3 kB)
Collecting pyparsing>=3 (from matplotlib)
  Downloading pyparsing-3.3.3-py3-none-any.whl.metadata (5.9 kB)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\vandana praj
apati\appdata\local\programs\python\python313\lib\site-packages (from matplo
tlib) (2.9.0.post0)
Requirement already satisfied: pandas>=1.2 in c:\users\vandana prajapati\app
data\local\programs\python\python313\lib\site-packages (from seaborn) (3.0.
6)
Collecting scipy>=1.10.0 (from scikit-learn)
  Downloading scipy-1.18.1-cp313-cp313-win_amd64.whl.metadata (61 kB)
Collecting joblib>=1.4.0 (from scikit-learn)
  Downloading joblib-1.6.0-py3-none-any.whl.metadata (6.5 kB)
Collecting narwhals>=2.0.1 (from scikit-learn)
  Downloading narwhals-2.26.0-py3-none-any.whl.metadata (15 kB)
Collecting threadpoolctl>=3.5.0 (from scikit-learn)
  Downloading threadpoolctl-3.7.0-py3-none-any.whl.metadata (24 kB)
Collecting cloudpickle>=3.0 (from joblib>=1.4.0->scikit-learn)
  Downloading cloudpickle-3.1.2-py3-none-any.whl.metadata (7.1 kB)
Requirement already satisfied: tzdata in c:\users\vandana prajapati\appdata
\local\programs\python\python313\lib\site-packages (from pandas>=1.2->seabor
n) (2026.4)
Requirement already satisfied: six>=1.5 in c:\users\vandana prajapati\appdat
a\local\programs\python\python313\lib\site-packages (from python-dateutil>=
2.7->matplotlib) (1.17.0)
Downloading matplotlib-3.11.2-cp313-cp313-win_amd64.whl (9.3 MB)
----- 0.0/9.3 MB ? eta -:-:-
-- ----- 0.5/9.3 MB 3.3 MB/s eta 0:00:03
----- 1.3/9.3 MB 3.5 MB/s eta 0:00:03
----- 2.1/9.3 MB 3.6 MB/s eta 0:00:03
----- 2.9/9.3 MB 3.6 MB/s eta 0:00:02
----- 3.4/9.3 MB 3.3 MB/s eta 0:00:02
----- 3.9/9.3 MB 3.1 MB/s eta 0:00:02

```

```

----- 4.5/9.3 MB 3.1 MB/s eta 0:00:02
----- 5.2/9.3 MB 3.2 MB/s eta 0:00:02
----- 5.8/9.3 MB 3.2 MB/s eta 0:00:02
----- 6.6/9.3 MB 3.2 MB/s eta 0:00:01
----- 7.1/9.3 MB 3.1 MB/s eta 0:00:01
----- 7.3/9.3 MB 3.1 MB/s eta 0:00:01
----- 7.9/9.3 MB 2.9 MB/s eta 0:00:01
----- 8.4/9.3 MB 2.9 MB/s eta 0:00:01
----- 8.9/9.3 MB 2.9 MB/s eta 0:00:01
----- 9.3/9.3 MB 2.8 MB/s eta 0:00:00
Downloading seaborn-0.13.2-py3-none-any.whl (294 kB)
Downloading networkx-3.7-py3-none-any.whl (2.1 MB)
----- 0.0/2.1 MB ? eta -:--:--
----- 0.3/2.1 MB ? eta -:--:--
----- 0.8/2.1 MB 2.3 MB/s eta 0:00:01
----- 1.6/2.1 MB 3.2 MB/s eta 0:00:01
----- 2.1/2.1 MB 3.6 MB/s eta 0:00:00
Downloading scikit_learn-1.9.1-cp313-cp313-win_amd64.whl (8.2 MB)
----- 0.0/8.2 MB ? eta -:--:--
----- 1.0/8.2 MB 5.2 MB/s eta 0:00:02
----- 2.1/8.2 MB 5.1 MB/s eta 0:00:02
----- 3.1/8.2 MB 5.2 MB/s eta 0:00:01
----- 3.9/8.2 MB 4.7 MB/s eta 0:00:01
----- 4.5/8.2 MB 4.4 MB/s eta 0:00:01
----- 5.2/8.2 MB 4.4 MB/s eta 0:00:01
----- 5.5/8.2 MB 4.3 MB/s eta 0:00:01
----- 5.5/8.2 MB 4.3 MB/s eta 0:00:01
----- 5.5/8.2 MB 4.3 MB/s eta 0:00:01
----- 6.3/8.2 MB 3.0 MB/s eta 0:00:01
----- 6.6/8.2 MB 2.9 MB/s eta 0:00:01
----- 6.6/8.2 MB 2.9 MB/s eta 0:00:01
----- 6.8/8.2 MB 2.6 MB/s eta 0:00:01
----- 7.1/8.2 MB 2.5 MB/s eta 0:00:01
----- 7.6/8.2 MB 2.4 MB/s eta 0:00:01
----- 7.9/8.2 MB 2.4 MB/s eta 0:00:01
----- 8.2/8.2 MB 2.3 MB/s eta 0:00:00
Downloading contourpy-1.4.0-cp313-cp313-win_amd64.whl (234 kB)
Downloading cycler-0.12.1-py3-none-any.whl (8.3 kB)
Downloading fonttools-4.66.0-cp313-cp313-win_amd64.whl (2.5 MB)
----- 0.0/2.5 MB ? eta -:--:--
----- 1.0/2.5 MB 5.4 MB/s eta 0:00:01
----- 2.1/2.5 MB 5.1 MB/s eta 0:00:01
----- 2.5/2.5 MB 5.0 MB/s eta 0:00:00
Downloading joblib-1.6.0-py3-none-any.whl (306 kB)
Downloading cloudpickle-3.1.2-py3-none-any.whl (22 kB)
Downloading kiwisolver-1.5.1-cp313-cp313-win_amd64.whl (70 kB)
Downloading narwhals-2.26.0-py3-none-any.whl (474 kB)
Downloading pillow-12.3.0-cp313-cp313-win_amd64.whl (7.2 MB)
----- 0.0/7.2 MB ? eta -:--:--
----- 1.0/7.2 MB 5.8 MB/s eta 0:00:02
----- 1.8/7.2 MB 4.9 MB/s eta 0:00:02
----- 2.6/7.2 MB 4.6 MB/s eta 0:00:02
----- 3.7/7.2 MB 4.8 MB/s eta 0:00:01
----- 5.0/7.2 MB 4.9 MB/s eta 0:00:01
----- 5.8/7.2 MB 4.9 MB/s eta 0:00:01
----- 6.6/7.2 MB 4.7 MB/s eta 0:00:01

```

```
----- 7.2/7.2 MB 4.5 MB/s eta 0:00:00
Downloading pyparsing-3.3.3-py3-none-any.whl (126 kB)
Downloading scipy-1.18.1-cp313-cp313-win_amd64.whl (36.6 MB)
----- 0.0/36.6 MB ? eta -:--:--
----- 0.8/36.6 MB 4.7 MB/s eta 0:00:08
----- 1.6/36.6 MB 4.6 MB/s eta 0:00:08
----- 2.1/36.6 MB 3.7 MB/s eta 0:00:10
----- 3.1/36.6 MB 3.8 MB/s eta 0:00:09
----- 3.9/36.6 MB 3.8 MB/s eta 0:00:09
----- 5.0/36.6 MB 4.1 MB/s eta 0:00:08
----- 6.0/36.6 MB 4.2 MB/s eta 0:00:08
----- 7.1/36.6 MB 4.4 MB/s eta 0:00:07
----- 7.9/36.6 MB 4.2 MB/s eta 0:00:07
----- 8.9/36.6 MB 4.4 MB/s eta 0:00:07
----- 10.0/36.6 MB 4.4 MB/s eta 0:00:0
6 ----- 11.3/36.6 MB 4.6 MB/s eta 0:00:0
6 ----- 12.3/36.6 MB 4.7 MB/s eta 0:00:0
6 ----- 13.9/36.6 MB 4.8 MB/s eta 0:00:0
5 ----- 15.2/36.6 MB 4.9 MB/s eta 0:00:0
5 ----- 16.5/36.6 MB 5.0 MB/s eta 0:00:0
4 ----- 17.3/36.6 MB 5.0 MB/s eta 0:00:0
4 ----- 18.6/36.6 MB 5.0 MB/s eta 0:00:0
4 ----- 19.7/36.6 MB 5.1 MB/s eta 0:00:0
4 ----- 21.0/36.6 MB 5.1 MB/s eta 0:00:0
4 ----- 22.0/36.6 MB 5.1 MB/s eta 0:00:0
3 ----- 23.1/36.6 MB 5.1 MB/s eta 0:00:0
3 ----- 24.1/36.6 MB 5.1 MB/s eta 0:00:0
3 ----- 25.4/36.6 MB 5.2 MB/s eta 0:00:0
3 ----- 26.2/36.6 MB 5.2 MB/s eta 0:00:0
3 ----- 27.3/36.6 MB 5.1 MB/s eta 0:00:0
2 ----- 28.3/36.6 MB 5.1 MB/s eta 0:00:0
2 ----- 28.3/36.6 MB 5.1 MB/s eta 0:00:0
2 ----- 29.4/36.6 MB 5.0 MB/s eta 0:00:0
2 ----- 30.4/36.6 MB 4.9 MB/s eta 0:00:0
2 ----- 31.5/36.6 MB 5.0 MB/s eta 0:00:0
2
```

1	-----	32.2/36.6 MB	4.9 MB/s	eta 0:00:0
1	-----	33.6/36.6 MB	4.9 MB/s	eta 0:00:0
1	-----	34.6/36.6 MB	4.9 MB/s	eta 0:00:0
1	-----	35.7/36.6 MB	5.0 MB/s	eta 0:00:0
1	-----	36.4/36.6 MB	5.0 MB/s	eta 0:00:0
1	-----	36.6/36.6 MB	4.9 MB/s	eta 0:00:0
0				

Downloading threadpoolctl-3.7.0-py3-none-any.whl (26 kB)

```
Installing collected packages: threadpoolctl, scipy, pyparsing, pillow, netw
orkx, narwhals, kiwisolver, fonttools, cyclcr, contourpy, cloudpickle, matpl
otlib, joblib, seaborn, scikit-learn
```

[illegible]

[illegible]

[illegible]

		5/15	[narwhals]
		5/15	[narwhals]
		5/15	[narwhals]
		5/15	[narwhals]
		5/15	[narwhals]
		5/15	[narwhals]
		5/15	[narwhals]
		5/15	[narwhals]
		5/15	[narwhals]
		5/15	[narwhals]
		6/15	[kiwisolver]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		7/15	[fonttools]
		8/15	[cycller]
		9/15	[contourpy]
		9/15	[contourpy]
		11/15	[matplotlib]
		11/15	[matplotlib]

[illegible]

[illegible]

```
In [17]: fraud_by_cat=df_train.groupby('category')['is_fraud'].mean().sort_values(asc
print(fraud by cat)
```

```

category
shopping_net      1.756149
misc_net          1.445795
grocery_pos       1.409761
shopping_pos      0.722538
gas_transport     0.469394
misc_pos          0.313853
grocery_net       0.294817
travel            0.286370
entertainment     0.247835
personal_care     0.242403
kids_pets         0.211439
food_dining       0.165098
home              0.160825
health_fitness    0.154869
Name: is_fraud, dtype: float64

```

```

In [23]: import networkx as nx
# Use a sample first (1.29M edges is a lot to visualize/test with)

sample=df_train.sample(50000,random_state=42)

G=nx.from_pandas_edgelist(
    sample,
    source='cc_num',
    target='merchant',
    edge_attr=['amt','is_fraud','trans_hour','category'],
    create_using=nx.MultiGraph() #MultiGraph since a customer can transact w
)

print("Number of nodes:",G.number_of_nodes())
print("number of edges:", G.number_of_edges())

```

```

Number of nodes: 1625
number of edges: 50000

```

```

In [25]: degrees=dict(G.degree())
print("Average degree:",sum(degrees.values())/len(degrees))
print('Max degree:', max(degrees.values()))

# which node has the most connections?

top_node= max(degrees, key=degrees.get)
print("Most connected node:", top_node, "with", degrees[top_node],"connections")

```

```

Average degree: 61.53846153846154
Max degree: 173
Most connected node: fraud_Kilback LLC with 173 connections

```

```

In [26]: # How many distinct merchants each customer has transacted with
customer_merchant_count = df_train.groupby('cc_num')['merchant'].nunique().reset_index()

# How many total transactions each customer has made
customer_txn_count = df_train.groupby('cc_num')['trans_num'].count().rename('txn_count').reset_index()

# Merge back into df_train
df_train = df_train.merge(customer_merchant_count, on='cc_num', how='left')

```

```
df_train = df_train.merge(customer_txn_count, on='cc_num', how='left')

print(df_train[['cc_num', 'customer_merchant_diversity', 'customer_txn_count']])
```

	cc_num	customer_merchant_diversity	customer_txn_count
0	2703186189652095	625	2028
1	630423337322	665	3030
2	38859492057661	347	503
3	3534093764340240	346	493
4	375534208663984	618	2017

```
In [27]: merchant_customer_count = df_train.groupby('merchant')['cc_num'].nunique().reset_index()
merchant_fraud_rate = df_train.groupby('merchant')['is_fraud'].mean().reset_index()

df_train = df_train.merge(merchant_customer_count, on='merchant', how='left')
df_train = df_train.merge(merchant_fraud_rate, on='merchant', how='left')

print(df_train[['merchant', 'merchant_customer_diversity', 'merchant_fraud_rate']])
```

	merchant	merchant_customer_diversity	merchant_fraud_rate
0	fraud_Rippin, Kub and Mann	609	0.014207
1	fraud_Heller, Gutmann and Zieme	784	0.010787
2	fraud_Lind-Buckridge	730	0.002111
3	fraud_Kutch, Hermiston and Farrell	757	0.003444
4	fraud_Keeling-Crist	666	0.003769

```
In [28]: # Sort by customer and time so rolling calculations make sense
df_train = df_train.sort_values(['cc_num', 'trans_date_trans_time']).reset_index()

# Set datetime as index temporarily for rolling window per customer
df_train['trans_count_last_1h'] = (
    df_train.groupby('cc_num')
    .apply(lambda x: x.set_index('trans_date_trans_time')['trans_num']
            .rolling('1h').count())
    .reset_index(level=0, drop=True)
    .values
)

print(df_train[['cc_num', 'trans_date_trans_time', 'trans_count_last_1h', 'is_fraud']])
```

	cc_num	trans_date	trans_time	trans_count_last_1h	is_fraud
0	180011453250192	2019-01-03	01:39:04	1.0	0
1	180011453250192	2019-01-03	05:51:22	1.0	0
2	180011453250192	2019-01-03	14:48:47	1.0	0
3	180011453250192	2019-01-04	10:39:10	1.0	0
4	180011453250192	2019-01-04	12:48:21	1.0	0
5	180011453250192	2019-01-04	18:56:05	1.0	0
6	180011453250192	2019-01-05	06:29:32	1.0	0
7	180011453250192	2019-01-05	08:15:39	1.0	0
8	180011453250192	2019-01-05	14:26:27	1.0	0
9	180011453250192	2019-01-05	15:05:38	2.0	0
10	180011453250192	2019-01-05	22:39:48	1.0	0
11	180011453250192	2019-01-07	02:27:23	1.0	0
12	180011453250192	2019-01-07	07:04:49	1.0	0
13	180011453250192	2019-01-07	18:03:11	1.0	0
14	180011453250192	2019-01-08	06:43:12	1.0	0

```
In [29]: print(df_train.groupby('is_fraud')['trans_count_last_1h'].describe())
```

	count	mean	std	min	25%	50%	75%	max
is_fraud								
0	1289169.0	1.186654	0.455767	1.0	1.0	1.0	1.0	7.0
1	7506.0	1.678124	0.954414	1.0	1.0	1.0	2.0	7.0

```
In [30]: # Merchant/customer diversity - recompute on test using test's own transacti
test_customer_merchant_count = df_test.groupby('cc_num')['merchant'].nunique
test_customer_txn_count = df_test.groupby('cc_num')['trans_num'].count().rer
test_merchant_customer_count = df_test.groupby('merchant')['cc_num'].nunique

df_test = df_test.merge(test_customer_merchant_count, on='cc_num', how='left')
df_test = df_test.merge(test_customer_txn_count, on='cc_num', how='left')
df_test = df_test.merge(test_merchant_customer_count, on='merchant', how='le

# merchant_fraud_rate: use TRAIN-derived rates only (avoid leakage)
df_test = df_test.merge(merchant_fraud_rate, on='merchant', how='left')
df_test['merchant_fraud_rate'] = df_test['merchant_fraud_rate'].fillna(df_tr

# Velocity feature for test set
df_test = df_test.sort_values(['cc_num', 'trans_date_trans_time']).reset_inc
df_test['trans_count_last_1h'] = (
    df_test.groupby('cc_num')
        .apply(lambda x: x.set_index('trans_date_trans_time')['trans_num']
            .rolling('1h').count())
        .reset_index(level=0, drop=True)
        .values
)

print(df_test.shape)
print(df_test[['cc_num', 'merchant', 'customer_merchant_diversity', 'merchant_f
```

```
(555719, 31)
      cc_num      merchant \
0  180011453250192      fraud_Brown Inc
1  180011453250192  fraud_McDermott, Osinski and Morar
2  180011453250192      fraud_Turner LLC
3  180011453250192  fraud_Jakubowski Group
4  180011453250192      fraud_Weber and Sons

      customer_merchant_diversity  merchant_fraud_rate  trans_count_last_1h
0                                474                0.002714                1.0
1                                474                0.001603                1.0
2                                474                0.001232                2.0
3                                474                0.001096                1.0
4                                474                0.001629                1.0
```

```
In [31]: feature_cols = [
        'amt', 'trans_hour', 'trans_day', 'trans_month', 'age',
        'city_pop', 'customer_merchant_diversity', 'customer_txn_count',
        'merchant_customer_diversity', 'merchant_fraud_rate', 'trans_count_last_1h'
    ]

X_train = df_train[feature_cols]
y_train = df_train['is_fraud']

X_test = df_test[feature_cols]
y_test = df_test['is_fraud']

print(X_train.shape, X_test.shape)
print(X_train.isnull().sum()) # sanity check for any leftover NaNs
```

```
(1296675, 11) (555719, 11)
amt                                0
trans_hour                        0
trans_day                        0
trans_month                      0
age                              0
city_pop                        0
customer_merchant_diversity      0
customer_txn_count              0
merchant_customer_diversity      0
merchant_fraud_rate              0
trans_count_last_1h             0
dtype: int64
```

```
In [32]: from sklearn.linear_model import LogisticRegression
        from sklearn.metrics import classification_report, roc_auc_score, average_precision_score

log_model = LogisticRegression(class_weight='balanced', max_iter=1000)
log_model.fit(X_train, y_train)

y_pred_log = log_model.predict(X_test)
y_proba_log = log_model.predict_proba(X_test)[:, 1]

print("=== Logistic Regression ===")
print(classification_report(y_test, y_pred_log))
```

```
print("ROC-AUC:", roc_auc_score(y_test, y_proba_log))
print("AUC-PR:", average_precision_score(y_test, y_proba_log))
```

C:\Users\VANDANA PRAJAPATI\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\linear_model_logistic.py:599: ConvergenceWarning: lbfgs failed to converge after 1000 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=1000).

You might also want to scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
=== Logistic Regression ===
```

	precision	recall	f1-score	support
0	1.00	0.93	0.96	553574
1	0.04	0.76	0.08	2145
accuracy			0.93	555719
macro avg	0.52	0.85	0.52	555719
weighted avg	1.00	0.93	0.96	555719

ROC-AUC: 0.8742453023401913

AUC-PR: 0.1532963978969879

In [34]: **from** sklearn.preprocessing **import** StandardScaler

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

In [35]: `log_model = LogisticRegression(class_weight='balanced', max_iter=1000)`
`log_model.fit(X_train_scaled, y_train)`

```
y_proba_log = log_model.predict_proba(X_test_scaled)[: , 1]
print("ROC-AUC:", roc_auc_score(y_test, y_proba_log))
print("AUC-PR:", average_precision_score(y_test, y_proba_log))
```

ROC-AUC: 0.8921286102009908

AUC-PR: 0.16453703126301977

In [36]: **from** sklearn.ensemble **import** RandomForestClassifier

```
rf_model = RandomForestClassifier(n_estimators=100, class_weight='balanced',
rf_model.fit(X_train, y_train)

y_pred_rf = rf_model.predict(X_test)
y_proba_rf = rf_model.predict_proba(X_test)[: , 1]

print("=== Random Forest ===")
print(classification_report(y_test, y_pred_rf))
print("ROC-AUC:", roc_auc_score(y_test, y_proba_rf))
```

```

=== Random Forest ===
              precision    recall  f1-score   support

     0       1.00      0.99      0.99     553574
     1       0.06      0.13      0.08       2145

 accuracy          0.99     555719
 macro avg          0.53     555719
 weighted avg       0.99     555719

```

ROC-AUC: 0.8869463402062476

```
In [37]: print("AUC-PR:", average_precision_score(y_test, y_proba_rf))
```

AUC-PR: 0.04264914270154089

```
In [38]: from sklearn.metrics import precision_recall_curve
import numpy as np

precisions, recalls, thresholds = precision_recall_curve(y_test, y_proba_rf)

for t in [0.1, 0.2, 0.3, 0.4, 0.5]:
    idx = np.argmin(np.abs(thresholds - t))
    print(f"Threshold={t}: Precision={precisions[idx]:.4f}, Recall={recalls[idx]:.4f}")
```

```

Threshold=0.1: Precision=0.0270, Recall=0.7744
Threshold=0.2: Precision=0.0266, Recall=0.6140
Threshold=0.3: Precision=0.0185, Recall=0.3832
Threshold=0.4: Precision=0.0141, Recall=0.1967
Threshold=0.5: Precision=0.0479, Recall=0.1310

```

```
In [39]: importances = pd.Series(rf_model.feature_importances_, index=feature_cols).sort_values(ascending=False)
print(importances)
```

```

amt                0.505728
trans_hour          0.173651
merchant_fraud_rate 0.107820
merchant_customer_diversity 0.043053
customer_merchant_diversity 0.042958
customer_txn_count   0.038510
trans_count_last_1h  0.034223
age                 0.022667
city_pop            0.015332
trans_month         0.009763
trans_day           0.006296
dtype: float64

```

```
In [40]: from math import radians

def haversine_distance(lat1, lon1, lat2, lon2):
    lat1, lon1, lat2, lon2 = map(np.radians, [lat1, lon1, lat2, lon2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat/2)**2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon/2)**2
    c = 2 * np.arcsin(np.sqrt(a))
    return 6371 * c # Earth radius in km

```

```
df_train['distance_km'] = haversine_distance(df_train['lat'], df_train['long'])
df_test['distance_km'] = haversine_distance(df_test['lat'], df_test['long'])

print(df_train.groupby('is_fraud')['distance_km'].describe())
```

	count	mean	std	min	25%	50%	\
is_fraud							
0	1289169.0	76.113756	29.119051	0.022255	55.332701	78.233012	
1	7506.0	76.268330	28.752602	0.738769	55.632890	77.931954	

	75%	max
is_fraud		
0	98.504498	152.117173
1	98.391090	144.522410

```
In [41]: # Customer's average and std transaction amount (computed only on train, to
customer_amt_stats = df_train.groupby('cc_num')['amt'].agg(['mean', 'std']).
        columns={'mean': 'customer_avg_amt', 'std': 'customer_std_amt'}
        )
customer_amt_stats['customer_std_amt'] = customer_amt_stats['customer_std_am

df_train = df_train.merge(customer_amt_stats, on='cc_num', how='left')
df_test = df_test.merge(customer_amt_stats, on='cc_num', how='left')
df_test[['customer_avg_amt', 'customer_std_amt']] = df_test[['customer_avg_am
df_train[['customer_avg_amt', 'customer_std_amt']].mean()
        )

# Z-score: how many std deviations is this transaction from the customer's r
df_train['amt_zscore'] = (df_train['amt'] - df_train['customer_avg_amt']) /
df_test['amt_zscore'] = (df_test['amt'] - df_test['customer_avg_amt']) / df_

# Also add a simple binary night flag, since hours 22-3 were clearly high-ri
df_train['is_night'] = df_train['trans_hour'].isin([22,23,0,1,2,3]).astype(int)
df_test['is_night'] = df_test['trans_hour'].isin([22,23,0,1,2,3]).astype(int)

print(df_train.groupby('is_fraud')['amt_zscore'].describe())
```

	count	mean	std	min	25%	50%	\
is_fraud							
0	1289169.0	-0.018656	0.938487	-1.088196	-0.395620	-0.180973	
1	7506.0	3.204177	3.320288	-2.825078	0.068244	2.255849	

	75%	max
is_fraud		
0	0.089414	52.889893
1	6.078607	15.789548

```
In [42]: feature_cols_v2 = [
        'amt', 'trans_hour', 'trans_day', 'trans_month', 'age',
        'city_pop', 'customer_merchant_diversity', 'customer_txn_count',
        'merchant_customer_diversity', 'merchant_fraud_rate', 'trans_count_last',
        'amt_zscore', 'is_night'
        ]

X_train = df_train[feature_cols_v2]
y_train = df_train['is_fraud']
X_test = df_test[feature_cols_v2]
```

```

y_test = df_test['is_fraud']

print(X_train.isnull().sum())
print(X_test.isnull().sum())

```

```

amt          0
trans_hour   0
trans_day     0
trans_month   0
age           0
city_pop      0
customer_merchant_diversity  0
customer_txn_count          0
merchant_customer_diversity  0
merchant_fraud_rate         0
trans_count_last_1h         0
amt_zscore     0
is_night       0
dtype: int64
amt          0
trans_hour   0
trans_day     0
trans_month   0
age           0
city_pop      0
customer_merchant_diversity  0
customer_txn_count          0
merchant_customer_diversity  0
merchant_fraud_rate         0
trans_count_last_1h         0
amt_zscore     0
is_night       0
dtype: int64

```

```

In [43]: from sklearn.preprocessing import StandardScaler
         from sklearn.linear_model import LogisticRegression
         from sklearn.metrics import classification_report, roc_auc_score, average_precision_score

         scaler = StandardScaler()
         X_train_scaled = scaler.fit_transform(X_train)
         X_test_scaled = scaler.transform(X_test)

         log_model = LogisticRegression(class_weight='balanced', max_iter=1000)
         log_model.fit(X_train_scaled, y_train)

         y_proba_log = log_model.predict_proba(X_test_scaled)[:, 1]
         print("=== Logistic Regression v2 ===")
         print("ROC-AUC:", roc_auc_score(y_test, y_proba_log))
         print("AUC-PR:", average_precision_score(y_test, y_proba_log))

=== Logistic Regression v2 ===
ROC-AUC: 0.9410757161370448
AUC-PR: 0.2271758581347975

```

```

In [44]: from sklearn.ensemble import RandomForestClassifier

         rf_model = RandomForestClassifier(n_estimators=100, class_weight='balanced',

```

```

rf_model.fit(X_train, y_train)

y_proba_rf = rf_model.predict_proba(X_test)[:, 1]
print("=== Random Forest v2 ===")
print("ROC-AUC:", roc_auc_score(y_test, y_proba_rf))
print("AUC-PR:", average_precision_score(y_test, y_proba_rf))

```

```

=== Random Forest v2 ===
ROC-AUC: 0.8894848775984812
AUC-PR: 0.06362088323449211

```

In [46]: `!pip install xgboost`

```

Requirement already satisfied: xgboost in c:\users\vandana prajapati\appdata\local\programs\python\python313\lib\site-packages (3.4.1)
Requirement already satisfied: numpy in c:\users\vandana prajapati\appdata\local\programs\python\python313\lib\site-packages (from xgboost) (2.5.3)
Requirement already satisfied: scipy in c:\users\vandana prajapati\appdata\local\programs\python\python313\lib\site-packages (from xgboost) (1.18.1)

```

```

[notice] A new release of pip is available: 25.1.1 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip

```

In [47]: `from xgboost import XGBClassifier`

```

# scale_pos_weight = ratio of negative to positive class, tells XGBoost how
scale_pos_weight = (y_train == 0).sum() / (y_train == 1).sum()
print("scale_pos_weight:", scale_pos_weight)

```

```

xgb_model = XGBClassifier(
    n_estimators=200,
    max_depth=6,
    learning_rate=0.1,
    scale_pos_weight=scale_pos_weight,
    eval_metric='aucpr',
    random_state=42,
    n_jobs=-1
)

xgb_model.fit(X_train, y_train)

y_proba_xgb = xgb_model.predict_proba(X_test)[:, 1]
print("=== XGBoost ===")
print("ROC-AUC:", roc_auc_score(y_test, y_proba_xgb))
print("AUC-PR:", average_precision_score(y_test, y_proba_xgb))

```

```

scale_pos_weight: 171.75179856115108
=== XGBoost ===
ROC-AUC: 0.8258108700434388
AUC-PR: 0.05967789514075205

```

In [48]: `for weight in [1, 10, 30, 171.75]:`

```

    xgb_model = XGBClassifier(
        n_estimators=200,
        max_depth=6,
        learning_rate=0.1,
        scale_pos_weight=weight,
        eval_metric='aucpr',

```

```

        random_state=42,
        n_jobs=-1
    )
    xgb_model.fit(X_train, y_train)
    y_proba = xgb_model.predict_proba(X_test)[: , 1]
    print(f"scale_pos_weight={weight}: ROC-AUC={roc_auc_score(y_test, y_proba)}")

```

```

scale_pos_weight=1: ROC-AUC=0.8042, AUC-PR=0.0154
scale_pos_weight=10: ROC-AUC=0.8442, AUC-PR=0.0286
scale_pos_weight=30: ROC-AUC=0.8360, AUC-PR=0.0289
scale_pos_weight=171.75: ROC-AUC=0.8258, AUC-PR=0.0597

```

```

In [49]: for col in feature_cols_v2:
        print(f"{col}: train_mean={X_train[col].mean():.4f}, test_mean={X_test[col].mean():.4f}")

```

```

amt: train_mean=70.3510, test_mean=69.3928
trans_hour: train_mean=12.8049, test_mean=12.8091
trans_day: train_mean=3.0706, test_mean=2.7268
trans_month: train_mean=6.1421, test_mean=9.5085
age: train_mean=45.5282, test_mean=46.4238
city_pop: train_mean=88824.4406, test_mean=88221.8879
customer_merchant_diversity: train_mean=580.6413, test_mean=424.1630
customer_txn_count: train_mean=1818.7253, test_mean=781.4784
merchant_customer_diversity: train_mean=719.9601, test_mean=503.0371
merchant_fraud_rate: train_mean=0.0058, test_mean=0.0058
trans_count_last_1h: train_mean=1.1895, test_mean=1.2243
amt_zscore: train_mean=-0.0000, test_mean=0.0036
is_night: train_mean=0.2351, test_mean=0.2342

```

```
In [50]: # Sort chronologically first
df_train = df_train.sort_values(['cc_num', 'trans_date_trans_time']).reset_index()
df_test = df_test.sort_values(['cc_num', 'trans_date_trans_time']).reset_index()

# Cumulative count of transactions per customer UP TO (and including) this time
df_train['customer_txn_count'] = df_train.groupby('cc_num').cumcount() + 1
df_test['customer_txn_count'] = df_test.groupby('cc_num').cumcount() + 1

# Cumulative distinct merchant count per customer up to this point
df_train['customer_merchant_diversity'] = (
    df_train.groupby('cc_num')['merchant']
    .transform(lambda x: x.groupby(x).cumcount() == 0) # marks first occurrence
)
df_train['customer_merchant_diversity'] = (
    df_train.groupby('cc_num')['customer_merchant_diversity'].cumsum()
)

df_test['customer_merchant_diversity'] = (
    df_test.groupby('cc_num')['merchant']
    .transform(lambda x: x.groupby(x).cumcount() == 0)
)
df_test['customer_merchant_diversity'] = (
    df_test.groupby('cc_num')['customer_merchant_diversity'].cumsum()
)

# Sanity check: compare distributions now
for col in ['customer_txn_count', 'customer_merchant_diversity']:
    print(f"{col}: train_mean={df_train[col].mean():.4f}, test_mean={df_test[col].mean():.4f}")

customer_txn_count: train_mean=909.8627, test_mean=391.2392
customer_merchant_diversity: train_mean=406.4752, test_mean=257.5761
```

```
In [51]: # Tag which dataset each row came from, then combine
df_train['is_train'] = 1
df_test['is_train'] = 0

df_all = pd.concat([df_train, df_test], ignore_index=True)
df_all = df_all.sort_values(['cc_num', 'trans_date_trans_time']).reset_index()

# Now compute cumulative features ONCE across the full combined timeline
df_all['customer_txn_count'] = df_all.groupby('cc_num').cumcount() + 1

df_all['_first_merchant_visit'] = (
    df_all.groupby('cc_num')['merchant']
    .transform(lambda x: x.groupby(x).cumcount() == 0)
)
df_all['customer_merchant_diversity'] = df_all.groupby('cc_num')['_first_merchant_visit'].cumsum()
df_all.drop(columns=['_first_merchant_visit'], inplace=True)

# Split back into train/test using the tag (preserves correct chronological order)
df_train = df_all[df_all['is_train'] == 1].drop(columns=['is_train']).reset_index()
df_test = df_all[df_all['is_train'] == 0].drop(columns=['is_train']).reset_index()

print(df_train.shape, df_test.shape)
```

```
for col in ['customer_txn_count', 'customer_merchant_diversity']:
    print(f"{col}: train_mean={df_train[col].mean():.4f}, test_mean={df_test
```

```
(1296675, 36) (555719, 36)
```

```
customer_txn_count: train_mean=909.8627, test_mean=2209.9887
```

```
customer_merchant_diversity: train_mean=406.4752, test_mean=605.0653
```

```
In [52]: # Re-tag and recombine (df_train/df_test already fixed for txn_count and mer
df_train['is_train'] = 1
df_test['is_train'] = 0
df_all = pd.concat([df_train, df_test], ignore_index=True)
df_all = df_all.sort_values(['cc_num', 'trans_date_trans_time']).reset_index

# Merchant diversity per customer (already correct from last step, skip recc

# Cumulative distinct customers per merchant (same logic, other direction)
df_all['_first_customer_visit'] = (
    df_all.groupby('merchant')['cc_num']
    .transform(lambda x: x.groupby(x).cumcount() == 0)
)
df_all['merchant_customer_diversity'] = df_all.groupby('merchant')['_first_c
df_all.drop(columns=['_first_customer_visit'], inplace=True)

# Expanding (cumulative, causal) mean and std of customer's own amount, EXCL
df_all['customer_avg_amt'] = (
    df_all.groupby('cc_num')['amt']
    .transform(lambda x: x.expanding().mean().shift(1))
)
df_all['customer_std_amt'] = (
    df_all.groupby('cc_num')['amt']
    .transform(lambda x: x.expanding().std().shift(1))
)
df_all['customer_avg_amt'] = df_all['customer_avg_amt'].fillna(df_all['amt'])
df_all['customer_std_amt'] = df_all['customer_std_amt'].fillna(1)

df_all['amt_zscore'] = (df_all['amt'] - df_all['customer_avg_amt']) / df_all

# Expanding (cumulative, causal) merchant fraud rate, EXCLUDING current tran
df_all['merchant_fraud_rate'] = (
    df_all.groupby('merchant')['is_fraud']
    .transform(lambda x: x.expanding().mean().shift(1))
)
df_all['merchant_fraud_rate'] = df_all['merchant_fraud_rate'].fillna(df_all[

# Split back
df_train = df_all[df_all['is_train'] == 1].drop(columns=['is_train']).reset_
df_test = df_all[df_all['is_train'] == 0].drop(columns=['is_train']).reset_i

print(df_train.shape, df_test.shape)
for col in ['merchant_customer_diversity', 'customer_avg_amt', 'customer_stc
    print(f"{col}: train_mean={df_train[col].mean():.4f}, test_mean={df_test
```

```
(1296675, 36) (555719, 36)
merchant_customer_diversity: train_mean=393.6833, test_mean=393.1535
customer_avg_amt: train_mean=70.4068, test_mean=70.0566
customer_std_amt: train_mean=131.8202, test_mean=141.8691
amt_zscore: train_mean=0.0350, test_mean=0.0144
merchant_fraud_rate: train_mean=0.0051, test_mean=0.0051
```

```
In [53]: feature_cols_v3 = [
    'amt', 'trans_hour', 'trans_day', 'trans_month', 'age',
    'city_pop', 'customer_merchant_diversity', 'customer_txn_count',
    'merchant_customer_diversity', 'merchant_fraud_rate', 'trans_count_last',
    'amt_zscore', 'is_night'
]

X_train = df_train[feature_cols_v3]
y_train = df_train['is_fraud']
X_test = df_test[feature_cols_v3]
y_test = df_test['is_fraud']

print(X_train.isnull().sum().sum(), X_test.isnull().sum().sum()) # should be 0 0
```

```
In [57]: from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.metrics import roc_auc_score, average_precision_score

# --- Logistic Regression ---
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

log_model = LogisticRegression(class_weight='balanced', max_iter=1000)
log_model.fit(X_train_scaled, y_train)
y_proba_log = log_model.predict_proba(X_test_scaled)[:, 1]
print("=== Logistic Regression v3 ===")
print("ROC-AUC:", roc_auc_score(y_test, y_proba_log))
print("AUC-PR:", average_precision_score(y_test, y_proba_log))

# --- Random Forest ---
rf_model = RandomForestClassifier(n_estimators=100, class_weight='balanced',
rf_model.fit(X_train, y_train)
y_proba_rf = rf_model.predict_proba(X_test)[:, 1]
print("=== Random Forest v3 ===")
print("ROC-AUC:", roc_auc_score(y_test, y_proba_rf))
print("AUC-PR:", average_precision_score(y_test, y_proba_rf))

# --- XGBoost ---
scale_pos_weight = (y_train == 0).sum() / (y_train == 1).sum()
xgb_model = XGBClassifier(
    n_estimators=200,
    max_depth=6,
    learning_rate=0.1,
    scale_pos_weight=scale_pos_weight,
```

```

    eval_metric='aucpr',
    random_state=42,
    n_jobs=-1
)
xgb_model.fit(X_train, y_train)
y_proba_xgb = xgb_model.predict_proba(X_test)[: , 1]
print("=== XGBoost v3 ===")
print("ROC-AUC:", roc_auc_score(y_test, y_proba_xgb))
print("AUC-PR:", average_precision_score(y_test, y_proba_xgb))

```

```

=== Logistic Regression v3 ===
ROC-AUC: 0.9363080947613459
AUC-PR: 0.20854237882091625
=== Random Forest v3 ===
ROC-AUC: 0.9587745225614779
AUC-PR: 0.6932839702037733
=== XGBoost v3 ===
ROC-AUC: 0.978713564492882
AUC-PR: 0.6318810234958433

```

```

In [58]: from sklearn.metrics import precision_recall_curve, classification_report
import numpy as np

precisions, recalls, thresholds = precision_recall_curve(y_test, y_proba_rf)

for t in [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7]:
    idx = np.argmin(np.abs(thresholds - t))
    print(f"Threshold={t}: Precision={precisions[idx]:.4f}, Recall={recalls[

```

```

Threshold=0.1: Precision=0.3955, Recall=0.7580
Threshold=0.2: Precision=0.6254, Recall=0.6662
Threshold=0.3: Precision=0.7585, Recall=0.5916
Threshold=0.4: Precision=0.8461, Recall=0.5459
Threshold=0.5: Precision=0.8921, Recall=0.4821
Threshold=0.6: Precision=0.9416, Recall=0.4135
Threshold=0.7: Precision=0.9733, Recall=0.3394

```

```

In [59]: importances = pd.Series(rf_model.feature_importances_, index=feature_cols_v3)
print(importances)

```

```

amt                                0.396827
amt_zscore                        0.152669
is_night                          0.129025
trans_hour                        0.100686
merchant_fraud_rate               0.051830
customer_merchant_diversity       0.032388
customer_txn_count                0.030594
trans_count_last_1h               0.026107
age                               0.023706
city_pop                          0.019378
merchant_customer_diversity       0.018597
trans_month                       0.010706
trans_day                         0.007490
dtype: float64

```

```

In [60]: y_pred_final = (y_proba_rf >= 0.2).astype(int)
print(classification_report(y_test, y_pred_final))

```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	553574
1	0.63	0.67	0.65	2145
accuracy			1.00	555719
macro avg	0.81	0.83	0.82	555719
weighted avg	1.00	1.00	1.00	555719

```
In [61]: import networkx as nx
import matplotlib.pyplot as plt

# Take a sample of fraud transactions and their surrounding legitimate trans
fraud_txns = df_train[df_train['is_fraud'] == 1].sample(30, random_state=42)

# Get the customers and merchants involved in these fraud cases
fraud_customers = fraud_txns['cc_num'].unique()
fraud_merchants = fraud_txns['merchant'].unique()

# Pull ALL transactions involving these specific customers/merchants (to sho
subgraph_data = df_train[
    (df_train['cc_num'].isin(fraud_customers)) | (df_train['merchant'].isin(
)].sample(min(2000, len(df_train)), random_state=42) # cap size for readabil

# Make sure our actual fraud sample is included
subgraph_data = pd.concat([subgraph_data, fraud_txns]).drop_duplicates(subse

print("Subgraph transactions:", len(subgraph_data))
print("Fraud transactions in subgraph:", subgraph_data['is_fraud'].sum())
```

Subgraph transactions: 2030
 Fraud transactions in subgraph: 50

```
In [62]: # Build the graph
G_sub = nx.from_pandas_edgelist(
    subgraph_data,
    source='cc_num',
    target='merchant',
    edge_attr=['amt', 'is_fraud'],
    create_using=nx.MultiGraph()
)

# Identify fraud edges for highlighting
fraud_edges = [(u, v) for u, v, d in G_sub.edges(data=True) if d.get('is_fra
normal_edges = [(u, v) for u, v, d in G_sub.edges(data=True) if d.get('is_fr

# Color nodes: customers vs merchants
customer_nodes = subgraph_data['cc_num'].unique()
merchant_nodes = subgraph_data['merchant'].unique()

plt.figure(figsize=(16, 12))
pos = nx.spring_layout(G_sub, k=0.3, seed=42, iterations=20)

# Draw normal edges (thin, gray, low opacity)
nx.draw_networkx_edges(G_sub, pos, edgelist=normal_edges, edge_color='lightc
```

```

# Draw fraud edges (thick, red)
nx.draw_networkx_edges(G_sub, pos, edgelist=fraud_edges, edge_color='red', w

# Draw nodes
nx.draw_networkx_nodes(G_sub, pos, nodelist=customer_nodes, node_color='steelblue',
nx.draw_networkx_nodes(G_sub, pos, nodelist=merchant_nodes, node_color='orange')

plt.title(f"Fraud Network Subgraph ({len(fraud_edges)} fraud edges highlighted in red)")
plt.legend(scatterpoints=1)
plt.axis('off')
plt.tight_layout()
plt.savefig('fraud_network_graph.png', dpi=150)
plt.show()

```

C:\Users\VANDANA PRAJAPATI\AppData\Local\Temp\ipykernel_24128\316799074.py:3
 4: UserWarning: Creating legend with loc="best" can be slow with large amounts of data.

```
plt.tight_layout()
```

C:\Users\VANDANA PRAJAPATI\AppData\Local\Temp\ipykernel_24128\316799074.py:3
 5: UserWarning: Creating legend with loc="best" can be slow with large amounts of data.

```
plt.savefig('fraud_network_graph.png', dpi=150)
```

C:\Users\VANDANA PRAJAPATI\AppData\Local\Programs\Python\Python313\Lib\site-packages\IPython\core\pylabtools.py:170: UserWarning: Creating legend with loc="best" can be slow with large amounts of data.

```
fig.canvas.print_figure(bytes_io, **kw)
```

Fraud Network Subgraph (50 fraud edges highlighted in red)

